RV College of Engineering®

AI Foundations for Engineers
CI124AT
UNIT 3

Prof. Nithyashree G D

Assistant Professor,

Department of Computer Science and Engineering,

RVCE

Nearest Neighbor Classifiers: Characteristics of Nearest Neighbour Classifiers.

Model Based v/s Instance Based

Go, change the world®

Model-Based Learning:

- Builds a **generalized model** from training data (rules, equations).
- Learns an **approximation of target function**.
- Once trained, predicts **without referring to training data**.
- **Example:** Spam filter learns rules to detect spam (e.g., suspicious words, unknown links).

Instance-Based Learning:

- **Does not build a model.** Retains all training data in memory.
- **Learning happens at prediction time.**
- Compares new input with stored instances using a **distance/similarity measure**.
- **Flexible but computationally expensive.**

Model Based v/s Instance Based

Key Features of Instance-Based Learning

- Memorizes all training examples → hence **memory-based**.
- Uses **similarity measures** (e.g., Euclidean distance) to find closest examples.
- Makes decisions based on **nearest or most similar instances**.
- **Lazy learner** → delays learning until a new query arrives.
- **No generalization rule** is created.

Example

- For a new email, the system checks similarity with stored spam/not-spam examples before deciding.

Classification Framework

•Involves two steps:

- **Inductive step:** Build a classification model from data.
- **Deductive step:** Apply model to classify test examples.

•Eager learners (e.g., Decision Trees, Rule-based):

- Build model immediately after training data is available.
- Note: Rule-Based Learning: A type of model-based learning where the model is represented as a set of IF–THEN rules.

•Lazy learners (e.g., k-NN):

- Delay model building until a test example needs classification.

How it Works

Each instance = a point in **d-dimensional feature space** (d = number of attributes).

For a test instance:

- Compute **distance/proximity** (e.g., Euclidean distance) to all training points.
- Identify **k-nearest neighbors** (1-NN, 2-NN, 3-NN, etc.).
- **Majority class** among neighbors → predicted label.

Example

•1-NN: Closest single point decides class.

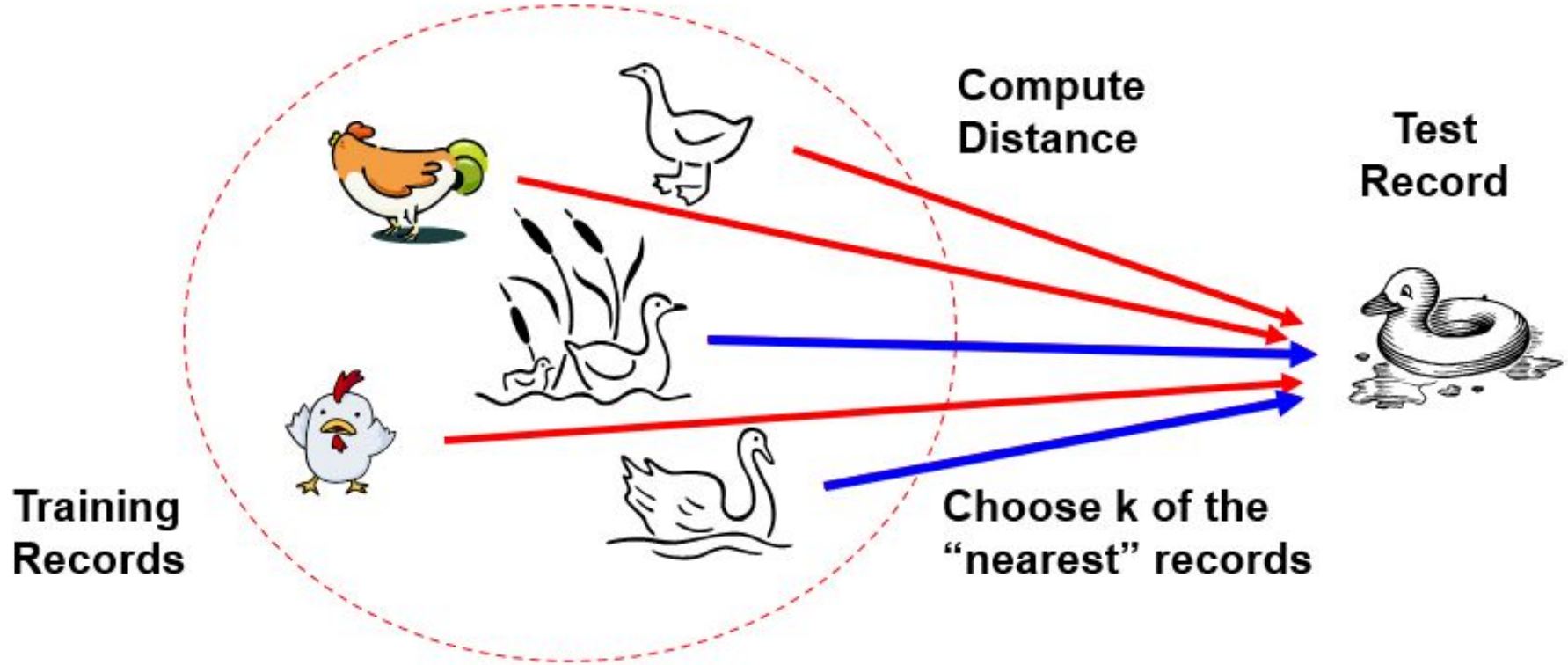
•3-NN: Majority among 3 closest points decides class.

Nearest Neighbour Classifiers

Go, change the world®

Basic idea:

If it walks like a duck, quacks like a duck, then it's probably a duck

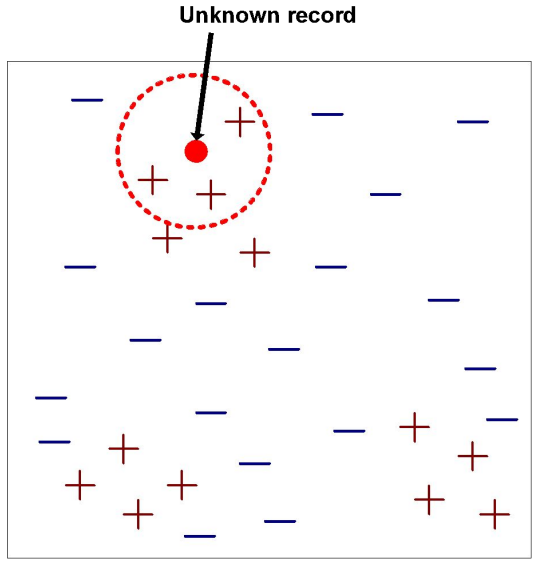


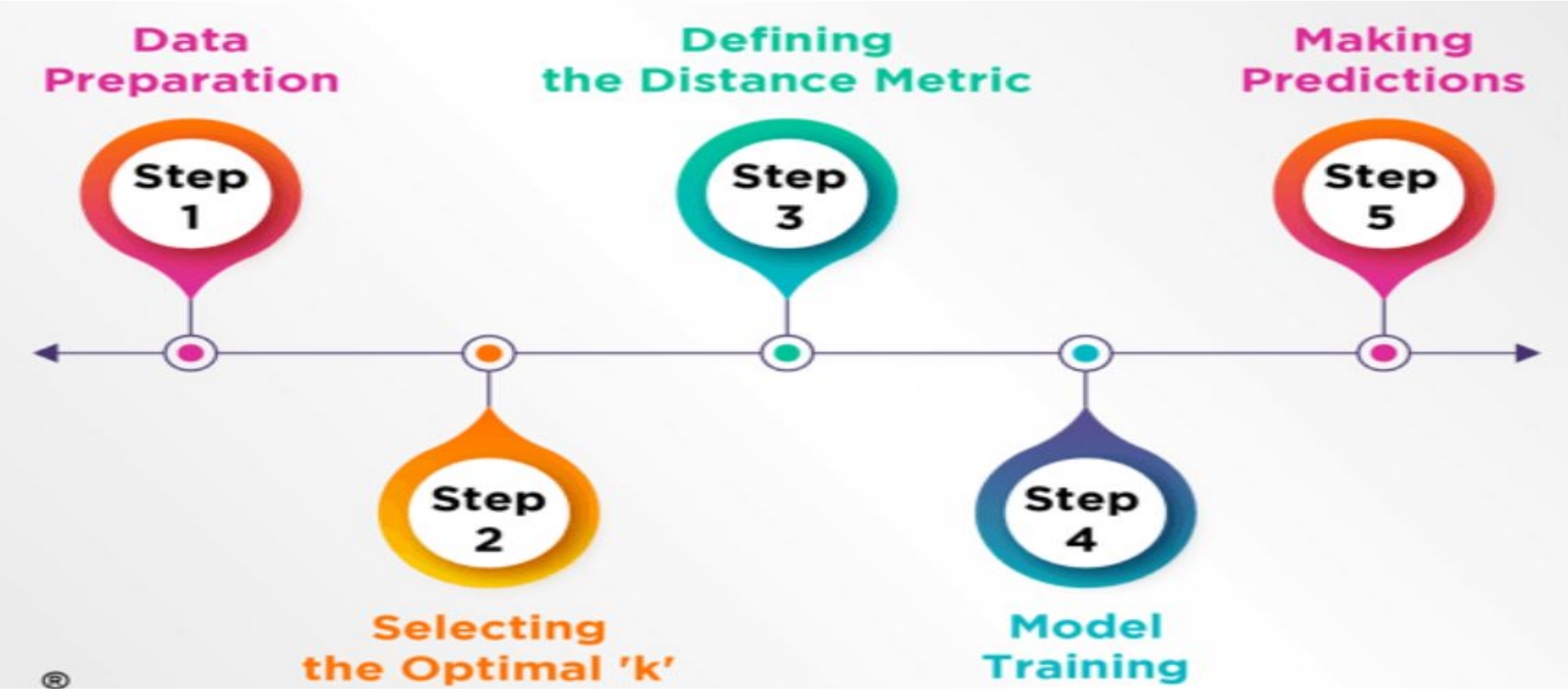
Nearest Neighbour Classifiers

Requires the following:

- A set of labeled records
- Proximity metric to compute distance/similarity between a pair of records

  e.g., Euclidean distance
- The value of *k*, the number of nearest neighbors to retrieve
- A method for using class labels of K nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)




Data Preparation
Step 1
Selecting the Optimal 'k'
Step 2
Defining the Distance Metric
Step 3
Model Training
Step 4
Making Predictions
Step 5

Characteristics of Nearest-Neighbor Classifiers (k-NN)

Go, change the world®

1. Instance-Based Learning

- Belongs to **instance-based learning** (no model abstraction).
- Uses **training instances directly** for prediction.
- Requires:
  - A **proximity/similarity measure** (e.g., Euclidean distance).
  - A **classification rule** (e.g., majority voting among nearest neighbors).

2. Lazy Learning Nature

- **No model-building phase** → minimal training time.
- **High prediction cost** → proximity must be computed for every test instance.
- Opposite of **eager learners** (e.g., decision trees), which train first and classify fast.

3. Local vs Global Decision

- Makes predictions using **local information** (neighbors of the test point).
- Decision trees/rule-based methods find **global patterns**.
- **Sensitive to noise**, especially for **small k** values.

Characteristics of Nearest-Neighbor Classifiers (k-NN)

Go, change the world®

4. Decision Boundaries

- A **decision boundary** separates one class from another in the feature space.
- KNN can form **complex, flexible, and non-linear boundaries**.
- It adapts to the **shape of the data**, not just straight lines.
- **More flexible** than decision trees, which create box-like (rectangular) boundaries.
- Boundaries **depend on training data** — small changes can alter results.

- **Small k** → very irregular boundary (overfitting).
- **Large k** → smoother boundary (better generalization).
- Increasing **k** helps reduce noise and makes predictions more stable.

5. Importance of Proper Distance Measure

- Accuracy depends on **choice of proximity metric** and **data scaling**.
- Attributes with different scales can distort distance (e.g., height in meters vs. weight in pounds).
- **Normalization/standardization** is essential.

K-Nearest-Neighbor Classifiers (k-NN)

Go, change the world®

The simplest instance based learning method is K Nearest Neighbor algorithm . KNN is a **non parametric algorithm** meaning that it does not make assumptions about shape of data and learns from data directly.

1. Compute the distance from the new data point to each point in the training set.
2. Choose the K closest points based on these distances.
3. Identify the most common class among these K neighbours.
4. Assign this majority class to the new data point.

K-Nearest-Neighbor Classifiers (k-NN)

Go, change the world®

Start

Choose K

Calculate the distance of new data point with all the existing data points

Sort the distances and select the k point with smallest distance

Average computation of surrounding k neighbors

Return the average

Finish

Ex: If we have a dataset of students with hours studied and marks scored in %, we want to classify a new student as Pass/Fail using KNN algorithm.

Study Hours	Marks Scored(%)	Class Labels
2	50	Fail
4	55	Fail
6	65	Pass
8	70	Pass
10	85	Pass

If the new data point is given as a student studying for 5 hours with marks scored 60%. What will be the predicted class for this student if K=3.

K-Nearest-Neighbor Classifiers (k-NN)

Go, change the world®

If the new data point is given as a student studying for 5 hours with marks scored 60%. What will be the predicted class for this student if K=3.

1) Using Euclidean distance to calculate distance between the new data point |5,60| to each point in the training set.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance to |2,50|:

$$d = \sqrt{(5-2)^2 + (60-50)^2} = \sqrt{109} = 10.44$$

Distance to |4,55|:

$$d = \sqrt{(5-4)^2 + (60-55)^2} = \sqrt{26} = 5.10$$

Distance to |6,65|:

$$d = \sqrt{(5-6)^2 + (60-65)^2} = \sqrt{26} = 5.10$$

Distance to |8,70|:

$$d = \sqrt{(5-8)^2 + (60-70)^2} = \sqrt{109} = 10.44$$

Distance to |10,85|:

$$d = \sqrt{5-10)^2 + (60-85)^2} = \sqrt{650} = 25.49$$

K-Nearest-Neighbor Classifiers (k-NN)

2) Sort the distances in ascending order

|4,55| : 5.10 - Fail

|6,65| : 5.10 - Pass

|2,50| : 10.44 - Fail

|8,70| : 10.44 - Pass

|10,85| : 25.49 - Pass

3) Choose K=3 closest neighbors

|4,55| : 5.10 - Fail

|6,65| : 5.10 - Pass

|2,50| : 10.44 - Fail

4) Find the majority class among these K neighbors

Fail - 2

Pass - 1

Majority class is Fail

The new student who studied for 5 hours and scored marks 60% is classified as Fail.

K-Nearest-Neighbor Classifiers (k-NN) Algorithm

Go, change the world®

Algorithm 5.2 The k-nearest neighbor classification algorithm.

1: Let k be the number of nearest neighbors and D be the set of training examples.
2: **for** each test example $z = (\mathbf{x}', y')$ **do**
3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
4: Select $D_z \subseteq D$, the set of k closest training examples to z.
5: $y' = \underset{v}{\text{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
6: **end for**

Distance Metrics

Go, change the world®

The idea behind KNN algorithm is based on the concept of “Nearness” or “Similarity”. The closer the new data point to the existing data, the more likely it is to belong to the same class as those nearby points.

To measure this nearness, we require some quantitative method, typically a distance metric. While there are many options to consider when we talk about distance calculation methods, **Euclidean Distance and Manhattan Distance** are widely used for KNN because of their simplicity, easy interpretation and its suitability to deal with numerical data.

The other distance metrics can be used in different context like **Cosine similarity** which is a better measurement metrics for problems that include higher dimensional sparse data or text data, **Hamming distance** which only works with binary or categorical data, **Chebyshev distance** which aids in board games or chess like grid problems, **Wasserstein (Earth Mover’s)** distance metrics which is used in Deep Learning concepts for comparing distributions(e.g, GANs), **Correlation distance** which can be used in time series or gene expression analysis to measure the dissimilarity, **Jaccard distance** that is used to measure the irregularities or dissimilarity in the sets most frequently used in the apps for recommendation system or clustering text data. The choice of right distance metrics impact performance of the algorithm.

(a) Euclidean Distance

Go, change the world®

The Euclidean distance is a **calculation of the shortest possible length between two points** which can be represented by a straight line if we connect them. The distance is always a non-negative number. The metric uses pythagorean theorem as its baseline to quantify the value.

The formula here depends on the number of dimensions. The features(columns) in the dataset are referred to as dimension here.

Each column defines one feature/dimension in the dataset. So if you have [“Age”, “Date-of-birth”], it is a 2D space. If it has three attributes, say [“Age”, “Height”, “Birthmark”], it becomes a 3D space. Depending on these numbers, we use different versions of Euclidean algorithm as below:

(a) Euclidean Distance

Go, change the world®

Two Dimensions (2D)

Given two points, (x_1, y_1) and (x_2, y_2), the Euclidean distance d is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Three Dimensions (3D)

Given two points, (x_1, y_1, z_1) and (x_2, y_2, z_2), the Euclidean distance d is given by:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2 + (z2 - z1)^2}$$

(a) Euclidean Distance

Go, change the world®

Here's the 2D graph showing Euclidean Distance between Point A (2, 1) and Point B (5, 4):

Euclidean Distance = $\sqrt{(5 - 2)^2 + (4 - 1)^2} = \sqrt{18} \approx$ **4.24**

(b) Manhattan Distance

(b) Manhattan Distance The Manhattan distance takes the coordinates of points and calculates the **sum of the absolute difference between the corresponding coordinates**.
It analyses and measures the distance for grid-like paths.

The formula for different dimensions are listed below:

Two Dimensions (2D)

Given two points: $A = (x_1, y_1)$ and $B = (x_2, y_2)$

Manhattan Distance (D):

$$D = |x_2 - x_1| + |y_2 - y_1|$$

(b) Manhattan Distance

Go, change the world®

Three Dimensions (3D)

Given two points: $A = (x_1, y_1, z_1)$ and $B = (x_2, y_2, z_2)$

Manhattan Distance (D): $D = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1|$

Imagine you're moving on a grid (like city streets) and you can only move **horizontally or vertically** — not diagonally. Let's visualize two points:

Point A: (2, 1)

Point B: (5, 4)

- Manhattan Distance = $|5 - 2| + |4 - 1| = 3 + 3 =$ **6**

How to Determine the Class Label of a Test Sample

Go, change the world®

When a new data point arrives:

1. Look through the **stored data in the memory unit** (called "instances") and check how similar each one is.
2. **Using this similarity, compare** new data to the closest matches and **predict** the output accordingly.

•**Find K nearest neighbors** – based on distance (like Euclidean, Manhattan, etc.).

Majority voting:

•If most neighbors belong to “Class A”, the new sample is labeled as “Class A.”

Weighted voting (optional):

•Closer points are more influential.

•Example:

$$w = \frac{1}{d^2}$$

means — if a neighbor is twice as far, it has only one-fourth the weight.

Choice of Proximity Measure Matters

Go, change the world®

Why it matters:

The **distance metric** determines what "closeness" means.

Choosing the wrong metric can completely change classification outcomes.

Common examples:

- **Euclidean Distance:** Straight-line distance; best for continuous numeric data.
- **Manhattan Distance:** Grid-like distance (sum of absolute differences).
- **Cosine Similarity:** Measures **angle** between vectors, not magnitude — perfect for text or high-dimensional sparse data.
    - Used in **document classification**, **NLP**, and **recommendation systems**.

Example to explain:

Two documents might have similar *words* (content direction) but different *lengths*.

Cosine similarity sees them as similar (since the angle between their word vectors is small), while Euclidean might not.

Data Preprocessing in KNN

Go, change the world®

Why preprocessing is critical:

KNN is **distance-based**, so features with large ranges dominate the calculation.
We need to make all features **comparable**.

Example:

- Height: 1.5 – 1.8
- Weight: 90 – 300
- Income: 10,000 – 1,000,000

If you use Euclidean distance, the difference in income overwhelms the height/weight — the algorithm becomes biased.

Solution:

- **Normalization:** Rescale features to a common range (e.g., 0–1).
- **Standardization (Z-score):** Transform data to have mean = 0 and SD = 1.

Analogy:

"It's like comparing distances in meters and kilometers — unless you convert them to the same unit, your comparison is meaningless."

Handling Missing Values

Go, change the world®

Problem:

KNN needs complete data for distance computation.

If even one attribute is missing, the distance cannot be calculated correctly.

Possible approaches:

1. **Imputation:** Fill missing values using:
    1. Mean / Median (for numeric data)
    2. Mode (for categorical data)
2. **Use partial attributes:**
    1. Compute distance only on attributes that exist in both instances.
    2. But this causes inconsistency because each pair might use a different subset → distances aren't comparable.

Handling Irrelevant and Redundant Attributes

Go, change the world®

Concept:

KNN treats **all features equally** in distance calculation.
So irrelevant or duplicated features can mislead the algorithm.

Problems:

- **Irrelevant features:** Add random noise → misclassification.
- **Redundant features:** Overemphasize certain patterns → bias in decision.

Example:

If you're classifying animals by size, color, and number of legs adding "serial number" as a feature adds *irrelevant noise*.
If you include both "weight" and "mass" — you've added *redundant information*.

Solution:

- **Feature Selection:** Use only discriminative features (e.g., mutual information, correlation analysis).
- **Dimensionality Reduction:** PCA (Principal Component Analysis) to remove redundancy.

K-NN Classifiers: Handling Interacting Attributes

Go, change the world®

Concept Overview

- **Feature interaction** occurs when the class label depends on a **combination of multiple attributes**, not on any single one.
- Individual features (like X or Y alone) may not separate the classes : the **joint relationship** matters.
- KNN can naturally handle such cases because it classifies based on **local neighborhood similarity** instead of fitting a global boundary.

Why Interaction Matters

- In many real datasets, attributes influence each other.
  Example: The result (class) depends on both *X* and *Y* together.
- The class distribution may form **complex or alternating patterns** (like checkerboards or interleaved clusters).
- Linear classifiers (e.g., logistic regression) **cannot separate** such non-linear patterns effectively.

K-NN Classifiers: Handling Interacting Attributes

Go, change the world®

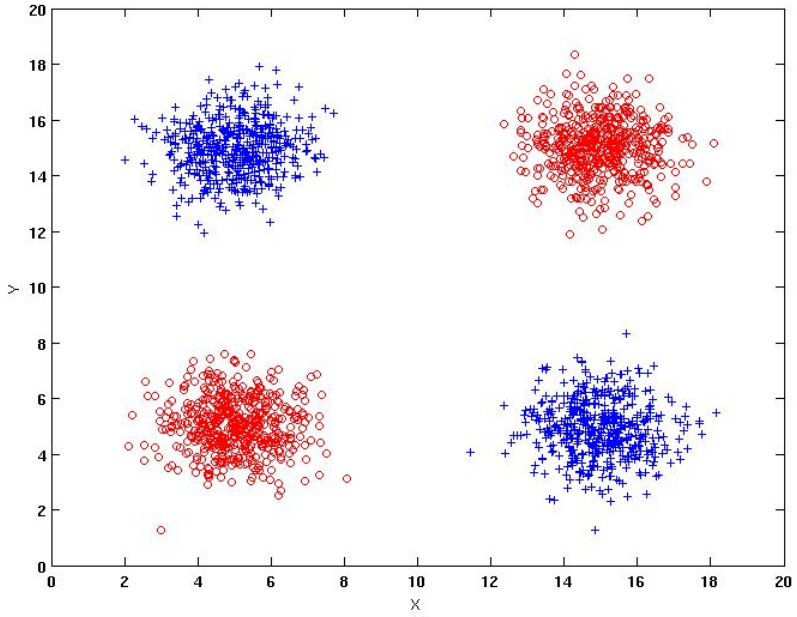
Illustrations

1.Cluster Pattern (First Figure):

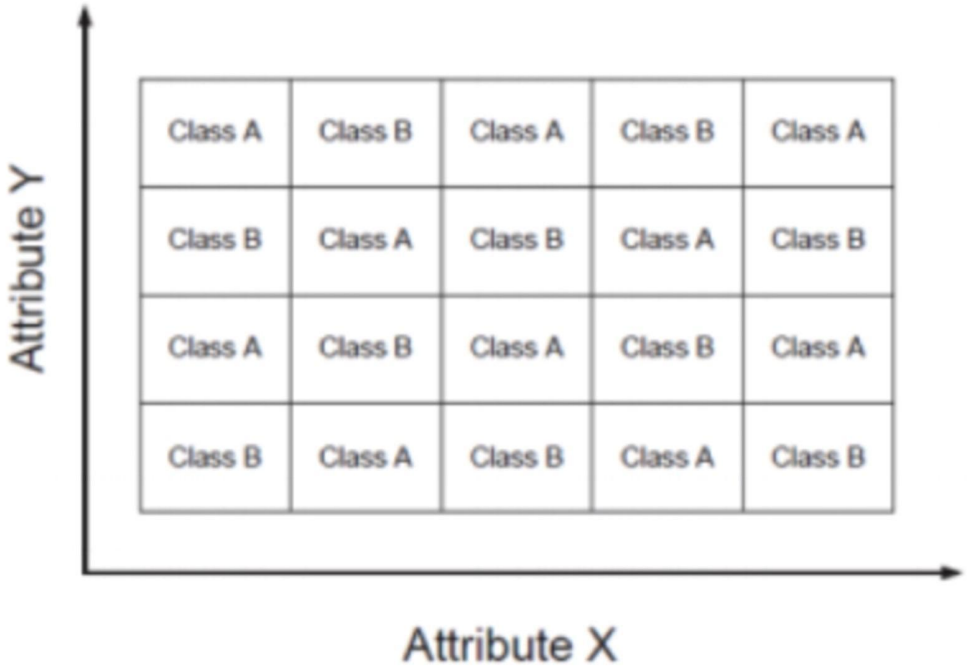
1. Four clusters of two classes (A and B) — separated diagonally.
2. Each class appears in multiple regions → shows attribute interaction.

2.Checkerboard Pattern (Second Figure):

1. Alternating Class A and Class B blocks → class changes based on *both* X and Y values.



· but KN



Choice of K and It's Impact

Go, change the world®

Meaning of K

- **K** = Number of nearest neighbors used for classification/prediction.
- Analogy: "Let me check with my **K closest friends** before deciding."
- The **choice of K** directly affects model performance.

K Value	Effect on Model
Small K	Model sensitive to noise → **high variance**
Large K	Model too smooth/generalized → **high bias**

Understanding Bias & Variance

Bias – Oversimplification

- The model is too *simple* to capture the real patterns in the data.
- It makes strong assumptions and ignores the finer details.
- As a result, it gives **systematic errors** — the same kind of mistake every time.
  - *Example:* Student uses one formula for all math problems.

Model misses important relationships → **underfitting**.

Variance:

- The model is too *complex* and reacts strongly to every detail or noise in training data.
- It fits the training data perfectly but performs poorly on new data.

Choice of K and It's Impact

Go, change the world®

Ideal Model

- Achieves **low bias** (captures patterns)
- And **low variance** (generalizes well)
- Balancing bias–variance is key to selecting the **optimal K**.

K value	Behavior	Impact on Performance	Analogy
Small K=1,3	Sensitive to noise	High variance, low bias	Relying on a single person's opinion- which could not be accurate
Moderate K=5,7,9	Captures local structure	Balanced trade-off	Asking a group of thoughtful people, thus getting good feedback
Large K=10,25,50	Includes irrelevant points	Low variance, high bias	Asking whole class with too many opinions- often not the one you were looking for

Choice of K and It's Impact

Go, change the world®

Scenario: Recommending study groups

If a student wants to join a particular study group based on interest and their availability.

K=1 joins a group with a single person, who might not be interested in that subject. Not a good choice.
K=3,5 joins a group with few people - might get good input.
K=20 joins a large group that may give a generic response not suiting your needs.

How to choose K:

1) Try different values of K to see which one gives the accurate result.
2) Choosing an odd number for K usually helps in avoiding ties.

Improving KNN Efficiency

Go, change the world®

KNN's biggest weakness:
It stores **all training data** and computes **distance with every sample** at prediction time → **slow for large datasets.**

Techniques to Improve Speed:

- **A. Efficient Data Structures**

1.k-d Trees:

•Divide data space into regions for faster neighbor search (best for low-dimension data).

2.Ball Trees / Metric Trees:

•Work better for medium to high dimensions.

3.Locality Sensitive Hashing (LSH):

•Groups nearby points into similar buckets using hash functions → used for high-dimensional or text data.

4.Approximate Similarity Search:

•Returns close-enough neighbors quickly — a good tradeoff between accuracy and speed.

Improving KNN Efficiency

Go, change the world®

- **B. Data Reduction**

1.Condensing:

- Keep a smaller subset of data that gives nearly the same accuracy.
- Example: Prototype selection.

2.Editing:

- Remove noisy or misclassified points → improves both accuracy and runtime.

Analogy:

"Instead of asking everyone in the city for advice (full dataset), you keep only the key representatives who think like the majority."